Verification Report: Non-Restoring Division Hardware Proofs

Automated Testbench Verification Suite

October 8, 2026

1 Algorithm Reference Overview

The hardware module implements a synchronous **Non-Restoring Division Algorithm**. This approach bypasses the intermediate subtraction roll-back penalty phase by evaluating the partial remainder's sign bit (MSB) continuously to decide whether the next step requires an addition or subtraction.

Step 1: Initialization

- $A = 00000$ (Accumulator register, sized to $n + 1$ bits)
- $M =$ Divisor (n bits)
- $Q =$ Dividend (n bits)
- count $= n$ (Loop counter tracking the bit-width)

Step 2: Main Operational Loop (Repeat n times)

1. **Shift Left:** Combined register pair $[A, Q]$ is shifted left by 1 bit position.
2. **Conditional Arithmetic Execution:**
   - **If** previous $A \geq 0$ (MSB of $A = 0$): Subtract M from A $(A = A - M)$
   - **Else** (MSB of $A = 1$): Add M to A $(A = A + M)$
3. **Quotient Bit Insertion (Q):**
   - **If** new calculation result $A \geq 0$: Set $Q_0 = 1$
   - **Else**: Set $Q_0 = 0$
4. **Decrement Counter:** count $=$ count $- 1$

Step 3: Final Correction Stage

- **If** final Accumulator $A < 0$: Perform remainder restoration: $A = A + M$

2 Hardware-Centric Step Proofs with Exact Waveform Timestamps

2.1 Test Case 1: $13 \div 3$

Parameters: Dividend $(Q) = 13$ (1101), Divisor $(M) = 3$ (0011 $\rightarrow -M = 11101$)
Initial Setup ($t = 40$ **ns**)**:** $A = 00000$, $Q = 1101$, count $= 4$

- **Iteration 1 (count** $= 4$**):**
  - **Sign Check** ($t = 60$ **ns**)**:** Previous A sign bit is positive (0).
  - **Shift & Arithmetic** ($t = 80$ **ns**)**:** Shift $[A, Q] \rightarrow A = 00001, Q = 101_$. $A = A - M \rightarrow 00001 + 11101 = 11110$ (Negative)
  - **Set Quotient Bit** ($t = 80$ **ns**)**:** New $A < 0 \rightarrow Q_0 = 0$. Resulting $Q = 1010$
  - **Wait / Loop Eval** ($t = 100$ **ns**)**:** Counter decremented (count $= 3$).
- **Iteration 2 (count** $= 3$**):**
  - **Sign Check** ($t = 120$ **ns**)**:** Previous A sign bit is negative (1).
  - **Shift & Arithmetic** ($t = 140$ **ns**)**:** Shift $[A, Q] \rightarrow A = 11101, Q = 010_$. $A = A + M \rightarrow 11101 + 00011 = 00000$ (Positive)
  - **Set Quotient Bit** ($t = 140$ **ns**)**:** New $A \geq 0 \rightarrow Q_0 = 1$. Resulting $Q = 0101$
  - **Wait / Loop Eval** ($t = 160$ **ns**)**:** Counter decremented (count $= 2$).
- **Iteration 3 (count** $= 2$**):**
  - **Sign Check** ($t = 180$ **ns**)**:** Previous A sign bit is positive (0).
  - **Shift & Arithmetic** ($t = 200$ **ns**)**:** Shift $[A, Q] \rightarrow A = 00000, Q = 101_$. $A = A - M \rightarrow 00000 + 11101 = 11101$ (Negative)
  - **Set Quotient Bit** ($t = 200$ **ns**)**:** New $A < 0 \rightarrow Q_0 = 0$. Resulting $Q = 1010$
  - **Wait / Loop Eval** ($t = 220$ **ns**)**:** Counter decremented (count $= 1$).
- **Iteration 4 (count** $= 1$**):**
  - **Sign Check** ($t = 220$ **ns**)**:** Previous A sign bit is negative (1).
  - **Shift & Arithmetic** ($t = 240$ **ns**)**:** Shift $[A, Q] \rightarrow A = 11010, Q = 010_$. $A = A + M \rightarrow 11010 + 00011 = 11101$ (Negative)
  - **Set Quotient Bit** ($t = 240$ **ns**)**:** New $A < 0 \rightarrow Q_0 = 0$. Resulting $Q = 0100$

Final Correction Stage ($t = 240$ **ns**)**:** Final A is negative (11101), restoration executed: $A = A + M \rightarrow 11101 + 00011 =$ **00001**.
Output Assertion ($t = 260$ **ns**)**: out_en** $= 1$**, Quotient** $(Q) =$ **4** (0100)**, Remainder** $(A) =$ **1** (0001)**.** **[MATCHES TESTBENCH]**

2.2 Test Case 2: $15 \div 3$

Parameters: Dividend $(Q) = 15$ (1111), Divisor $(M) = 3$ (0011 $\rightarrow -M = 11101$)
Initial Setup ($t = 340$ **ns**)**:** $A = 00000$, $Q = 1111$, count $= 4$

- **Iteration 1 (count** $= 4$**):**
  - **Sign Check** ($t = 360$ **ns**)**:** Previous $A \geq 0$.
  - **Shift & Arithmetic** ($t = 380$ **ns**)**:** Shift $[A, Q] \rightarrow A = 00001, Q = 111_$. $A = A - M \rightarrow 00001 + 11101 = 11110$ (Negative) $\rightarrow Q_0 = 0$. Resulting $Q = 1110$

- **Iteration 2 (count = 3):**
  - **Sign Check ($t = 400$ ns):** Previous $A < 0$.
  - **Shift & Arithmetic ($t = 420$ ns):** Shift $[A, Q] \rightarrow A = 11101, Q = 110_$. $A = A + M \rightarrow 11101 + 00011 = 00000$ (Positive) $\rightarrow Q_0 = 1$. Resulting $Q = 1101$
- **Iteration 3 (count = 2):**
  - **Sign Check ($t = 440$ ns):** Previous $A \geq 0$.
  - **Shift & Arithmetic ($t = 460$ ns):** Shift $[A, Q] \rightarrow A = 00001, Q = 101_$. $A = A - M \rightarrow 00001 + 11101 = 11110$ (Negative) $\rightarrow Q_0 = 0$. Resulting $Q = 1010$
- **Iteration 4 (count = 1):**
  - **Sign Check ($t = 480$ ns):** Previous $A < 0$.
  - **Shift & Arithmetic ($t = 500$ ns):** Shift $[A, Q] \rightarrow A = 11101, Q = 010_$. $A = A + M \rightarrow 11101 + 00011 = 00000$ (Positive) $\rightarrow Q_0 = 1$. Resulting $Q = 0101$

Final Correction Stage ($t = 520$ ns): Final Accumulator $A = 00000 \geq 0$. No restoration required.
Output Assertion ($t = 540$ ns): out_en = 1, Quotient (Q) = 5 (0101), Remainder (A) = 0 (0000). [MATCHES TESTBENCH]

2.3 Test Case 3: $9 \div 4$

Parameters: Dividend (Q) = 9 (1001), Divisor (M) = 4 ($0100 \rightarrow -M = 11100$)
Initial Setup ($t = 620$ ns): $A = 00000$, $Q = 1001$, count = 4

- **Iteration 1 (count = 4):**
  - **Sign Check ($t = 640$ ns):** Previous $A \geq 0$.
  - **Shift & Arithmetic ($t = 660$ ns):** Shift $[A, Q] \rightarrow A = 00001, Q = 001_$. $A = A - M \rightarrow 00001 + 11100 = 11101$ (Negative) $\rightarrow Q_0 = 0$. Resulting $Q = 0010$
- **Iteration 2 (count = 3):**
  - **Sign Check ($t = 680$ ns):** Previous $A < 0$.
  - **Shift & Arithmetic ($t = 700$ ns):** Shift $[A, Q] \rightarrow A = 11010, Q = 010_$. $A = A + M \rightarrow 11010 + 00100 = 11110$ (Negative) $\rightarrow Q_0 = 0$. Resulting $Q = 0100$
- **Iteration 3 (count = 2):**
  - **Sign Check ($t = 720$ ns):** Previous $A < 0$.
  - **Shift & Arithmetic ($t = 740$ ns):** Shift $[A, Q] \rightarrow A = 11100, Q = 100_$. $A = A + M \rightarrow 11100 + 00100 = 00000$ (Positive) $\rightarrow Q_0 = 1$. Resulting $Q = 1001$
- **Iteration 4 (count = 1):**
  - **Sign Check ($t = 760$ ns):** Previous $A \geq 0$.
  - **Shift & Arithmetic ($t = 780$ ns):** Shift $[A, Q] \rightarrow A = 00001, Q = 001_$. $A = A - M \rightarrow 00001 + 11100 = 11101$ (Negative) $\rightarrow Q_0 = 0$. Resulting $Q = 0010$

Final Correction Stage ($t = 820$ ns): Final A is negative (11101), restoration executed: $A = A + M \rightarrow 11101 + 00100 = \mathbf{00001}$.
Output Assertion ($t = 840$ ns): out_en = 1, Quotient (Q) = 2 (0010), Remainder (A) = 1 (0001). [MATCHES TESTBENCH]